

What Belongs in an Agent's Context Window?

NEIL LUNAVAT, Pimpri Chinchwad College of Engineering, India

An LLM coding agent works inside a finite context window, and before it edits a file of an unfamiliar repository it must choose what else to put there. Given the file being changed, we compare the two signals tools use for that choice — proximity in the import graph and the words of the issue report — and their fusion, on 2,412 change tasks from 1,162 pull requests in 112 Python repositories, scoring each reading order by how much of an answer key it covers per line read.

Our main contribution is a measurement protocol. Benchmarks built from issue reports mix two regimes, because in two tasks in five the issue names a file the change touches. Stratifying by that naming, comparing against an issue-free twin of each ordering, and deleting only the names from the same issue separates them. Applied here, it shows the issue is worth +0.156 AUC to the best ordering where it names a key file against a non-significant +0.016 elsewhere, and that about half of that difference is the naming itself; a dense code retriever reads the issue better but leaves the structure intact. Under the same controls the apparent contest between graph and text dissolves, and a key annotated by people sides with the patch-derived key the field already uses. The practical recommendation is to fuse a walk from the file with issue retrieval: it held under predictions committed before scoring on seven unseen repositories and on the human-annotated key, and when the file to change must itself be guessed. Everything here runs without paid model access; we specify, but do not run, the end-to-end agent experiment.

Additional Key Words and Phrases: LLM agents, context windows, code search, context retrieval, dependency graphs, benchmark validity, empirical software engineering

ACM Reference Format:

Neil Lunavat. 2026. What Belongs in an Agent's Context Window?. *ACM Trans. Softw. Eng. Methodol.* 1, 1 (September 2026), 22 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

An LLM coding agent is handed a bug report and a file to change. Before it can write anything it must decide what else to put in its context window, and it cannot simply include everything. In the corpus we study the median pull request is made against a repository of 286,385 tokens — counting the Python files in the import graph, excluding tests and non-Python files — and 74.8% of pull requests are made against a repository larger than a 128,000-token window. The files that carry the answer are a median 2.98% of the repository. Something has to choose.

Two signals are available. The dominant one in tool building is the dependency graph: Aider builds a repository map from definitions and references, and RepoGraph, LocAgent and CoSIL walk a code graph outward from a starting point, on the premise that what you need is near what you are changing. The other is the issue report, which SWE-bench's own retrieval baseline queries with BM25 [10] and which every agent reads. We compare the two, and their fusion, as *reading orders* scored by how much of an answer key they cover per line read. Every task hands the reader the file being changed; the question is what *else* to read. We measure retrieval, not repair.

A naive version of this comparison is easy to run and misleading. Graded against the files co-edited in the pull request, issue text wins; graded against the files defining symbols the change uses, structure wins, and the method ranking nearly inverts. Neither half survives two controls. The structural half is a population effect: a symbol key exists for single-file changes and a co-edited key does not, so the two are scored on different pull requests. The lexical half is largely the issue

Author's address: Neil Lunavat, Pimpri Chinchwad College of Engineering, India, neillunavat3192@gmail.com.

2026. 1049-331X/2026/9-ART \$15.00
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

naming the answer: in two tasks in five the issue names a changed file as code, which a method matching issue words against file paths can simply read. A key annotated by people then sides with the co-edited key.

What survives is two findings, and the paper is organised around them. The first concerns the benchmark more than the methods. Issue-aware retrieval is worth far more on issues that name their files, and a within-task redaction experiment measures how much of that is the names themselves. We call the names *leakage* only in this benchmark sense: in deployment an issue that names `core.py` is a legitimate input, but an average over issues that do and do not describes neither, and the mix varies by corpus. The second is a recommendation: fusing a walk from the file with issue retrieval is robust across keys, strata, unseen repositories, a human-annotated key, alternative cost rules and the loss of the free seed, except for single-file changes, where the walk alone is better.

Research questions.

RQ1 Does the uncontrolled comparison of structural and lexical reading orders survive matching the populations and controlling for issues that name the answer?

RQ2 What is issue text worth to retrieval, and how much of it is the issue naming the files to change?

RQ3 Which ordering is robust, and does it hold on unseen repositories, a human-annotated key, other cost rules, and without a free seed?

RQ4 What, beyond that, does the import graph contribute?

Contributions.

- (1) **A protocol for measuring what issue text is worth.** An issue-free twin of each ordering gives the issue's total value, and three nested within-task redactions give the part carried by what points at the key files. On issues that name a key file as code the issue is worth +0.156 AUC to the best fusion, of which 0.061 to 0.078 is attributable to the names; on the rest it is worth +0.016. A dense retriever reads the issue better but the structure is the same (Section 4.2).
- (2) **A robust recommendation, tested out of sample.** Reciprocal-rank fusion of a personalised walk with issue retrieval is never significantly worse than the best pure ordering on the matched population, held under predictions committed before scoring on seven unseen repositories and on ContextBench's human-annotated key, and remains best without a free seed. Whether the walk should favour short files depends on how a context is charged (Section 4.3).
- (3) **Controls that changed conclusions here:** matching the populations two keys are scored on; stratifying by whether the issue names a key file as code; and the repository as the unit, which leaves the headline comparisons significant but changes the significance verdict on 102 of the 1,076 comparisons significant with the pull request as the unit (none changes sign).
- (4) **Corrections to common arguments for graph retrieval.** The two-hop neighbourhood holds on average 87.5% of the needed files and 56% of the repository. Walking against import edges is costly, discarding direction is not. And the advantage structure shows on a key built from imports is not corroborated by a human-annotated key (Section 4.4).

2 RELATED WORK

Structure and history as guides to change. Impact analysis [4] propagates from a changed element along dependencies; Robillard [19] ranks program elements by dependency topology; feature-location surveys [7] note that structural, textual and dynamic techniques are usually evaluated

Table 1. What repository-context systems are graded against, read from each paper’s own evaluation section (2026-09-23).

System	Venue	Ground truth used for evaluation
<i>Files touched by the gold patch</i>		
LocAgent [6]	ACL 2025	“The affected files or functions ...as identified in the patches, are considered the target locations”
CoSIL [9]	ASE 2025	File- and function-level accuracy on SWE-bench Lite and Verified, i.e. the patched locations
Agentless [21]	FSE 2025	Correct if the edit covers a superset of the ground-truth patch locations
LingmaAgent [15]	FSE 2025	Fault locations extracted from the developer patch
SWE-Fixer [22]	ACL 2025	Precision and recall against the defective files of the resolving pull request
KGCompass [23]	2025	Files and code elements modified by the ground-truth patch
SWE-Adept [8]	2026	Correct only if all patched locations are in the top- <i>k</i>
SWERank [18]	2025	Every function modified in the pull request
CodeScout [20]	2026	LocAgent’s patch-derived targets
RepoGraph [16]	ICLR 2025	Edit locations of the ground-truth patch
<i>Something other than the gold patch alone</i>		
GraphLocator [14]	2025	Patch locations, dual-annotated because patch-derived ground truth “contains missing locations”
SWE-Explore [26]	2026	Code regions that successful agents actually read
ContextBench [12]	2026	Human-annotated context needed to reconstruct the fix
AgentRetrievalBench [17]	2026	Files whose reading helps the agent’s next step
CodeR [5]	2024	Golden locations marked by the authors

in isolation; and Ko et al. [11] observed developers navigating largely by names and search. Co-change mining [24, 27] suggests files that change together; our co-edited key uses that notion as ground truth rather than as a method.

Graph-based context for agents. SWE-bench [10] and its successors [3, 25] made repository-scale change tasks a standard benchmark, and a family of systems assembles context by walking a code graph: Aider’s repository map [1], RepoGraph [16], LocAgent [6] and CoSIL [9]; Agentless [21] leans on file and directory names instead. Table 1 records what each is graded against, read from its own evaluation section. Almost all use the files touched by the gold patch — our co-edited key — and our results support that choice: of our two proxies it is the one a human-annotated key agrees with. What none of them reports is its results separately for issues that do and do not name the files being scored.

Issues that give the answer away. SWE-Bench+ [2] finds that 32.67% of the patches SWE-Agent with GPT-4 gets passed had the solution in the issue or its comments, and Agentless reports issues containing the exact patch. A weaker form concerns only *where*: a retrieval step needs only to know which files to open. This too is known. Agentless classifies the location information in SWE-bench Lite and finds that around half of its issues already provide the location of the file to be edited, and the SWE-Bench Illusion study [13] shows that models name the buggy file from the issue alone in up to 76% of SWE-bench Verified tasks against 53% on repositories outside SWE-bench. Our rate, 40.2% of tasks naming a key file as code, is in line with the first. What we add is not the rate but its consequence: how much of an issue-aware method’s score it accounts for, separated within task from the rest of what the issue says.

Concurrent work on the key. SWE-Explore [26] derives ground truth from code that successful agents read and scores ranked lists under a line budget, close to our cost model; ContextBench [12]

annotates gold context by hand; AgentRetrievalBench [17] scores files by whether they help the next step; GraphLocator [14] dual-annotates the patch key. We use ContextBench’s key as a third, independent answer key (Section 4.3).

3 STUDY DESIGN

The corpus, graph builder, task definition and metric were fixed and tagged in version control before any method was run (the *design freeze*); the method families, the fusion among them, were added afterwards in response to the first results, so the fusion began as an exploratory result. We then tested it twice under predictions committed to the repository before any of the data they concern were scored (paper/external_predictions.md, paper/contextbench_predictions.md). A signed tag in our own repository establishes order only to a reader who trusts the repository; we do not call either step a pre-registration. Analyses added in response to review and not fixed in advance — the cost rules, the dense retriever, the systems’ selectors and the condition without a free seed — are marked exploratory where they appear, and every change made after results existed is logged in the replication package.

3.1 Corpus, tasks and keys

We drew 1,886 pull requests from SWE-bench, SWE-bench-Live and SWE-rebench in Python repositories that pass our sampling rules, at most 20 per repository. A pull request that edits k existing non-test source files that are nodes of the import graph yields k tasks: each edited file in turn becomes the *seed*, and the task is to order the remaining files of the repository. This gives 3,123 tasks from 1,873 pull requests in 113 repositories (13 pull requests edit no graph file). Of these, 2,412 tasks — 1,162 pull requests in 112 repositories — have at least one key file once the seed is removed, and they are the ones scored. Of the 711 unscored tasks, 706 come from pull requests changing a single source file; they are 61.2% of all 1,154 such tasks, because the symbol key below is empty whenever the changed code resolves to nothing outside itself. The median task’s repository has 131 graph nodes and the median key 2 files.

Graphs are built per base commit with an ast walker: nodes are source files, an edge $a \rightarrow b$ means a imports a module resolving to b , test files are excluded, and per-file line counts from the same pass are the cost unit. Two keys, never merged:

co_edited The other non-test source files changed by the same pull request: what a developer in fact touched, independent of the import graph. Available only when a pull request edits more than one graph file: 1,958 tasks from 708 pull requests.

symbol Files defining names used on the lines the pull request adds, resolved by jedi’s goto with follow_imports=True on the patched code: what the change depends on. 2,341 tasks from 1,156 pull requests.

The symbol key is built by following imports, so it is partly circular with the structural orderings, and we treat it as the secondary proxy. The 702 pull requests carrying both keys form the *matched population*, on which every cross-key comparison is repeated. Of the 1,156 pull requests with a symbol key, 454 have no co-edited key because they edit a single graph file; we call these *single-file* pull requests. By *issue text* we mean the problem statement only, not maintainers’ hints_text.

A third key, used only in Section 4.3, is ContextBench’s human-annotated gold context [12]: the Python files a person judged necessary to reconstruct the fix, less the seed. We ran its SWE-bench Verified and SWE-bench Pro Python tasks through the unchanged pipeline, drawn by our sampling rules: 108 pull requests in 14 repositories have a non-empty gold key, 6 of them also in the main sample.

Table 2. Reading orders. Structural orderings use the seed and the graph; global orderings neither the seed nor the issue; lexical orderings no graph.

Family	Method	Ordering
structure	bfs_und, hops_lines	breadth-first from the seed; by (hops, file length)
	ppr_und, ppr_und_pl	personalised PageRank from the seed, undirected
	ppr_out, ppr_out_pl	the same walk along import edges: what the seed uses
	ppr_in, ppr_in_pl	the same walk against them: what uses the seed
global	pagerank, indegree	repository prior; ignores seed and issue
lexical	bm25_seed, path_seed	BM25 over contents or paths, queried by the seed
	bm25_issue, bm25_issue_pl	BM25 over contents, queried by the issue
	path_issue	BM25 over file paths, queried by the issue
fusion	rff_hops_path	RRF of hops_lines and path_issue
	rff_hops_issue_path	RRF of hops_lines, bm25_issue, path_issue
	rff_pprpl_issue_path	RRF of ppr_und_pl, bm25_issue, path_issue
	rff_ppr_issue, rff_ppr_issue_path	the same on the unscaled ppr_und
reference	same_dir, random, oracle	seed's directory first; shuffled; key files only, cheapest first

3.2 Reading orders

Every ordering receives the graph, the seed, per-file identifier bags and the issue text, and ranks every other file (Table 2). Identifier bags split identifiers on underscores and camel case; the issue is tokenised the same way. BM25 uses $k_1 = 1.5$, $b = 0.75$ and differs between methods only in the query. Fusion is reciprocal-rank fusion (RRF) with $k = 60$. Variants suffixed *_pl* divide a score by file length. Four orderings recur, and in the text we use plain names for them: *the fusion* (rff_pprpl_issue_path), *the length-aware walk* (ppr_und_pl), *the hop walk* (hops_lines) and *path matching* (path_issue). Structural orderings are handed a correct seed; issue orderings get no seed at all.

Three further kinds of ordering were added after review and are scored but kept out of the ranks and correction families of the other tables, so that adding them moves nothing reported elsewhere. A *dense retriever*, jina-embeddings-v2-base-code — the model our design fixed before any method ran — ranks files by cosine similarity of their first 512 tokens to the issue (dense_issue), which is embedded whole as the length-weighted mean of its 512-token chunks; its fusions replace or join BM25 over contents. The *issue-free twins* of the fusions replace every issue-derived input by its seed-derived counterpart. And the *context selectors of three published systems* — Aider's repository map, RepoGraph's two-hop ego graph and LocAgent's typed breadth-first search, reimplemented from their papers — are scored as orderings, with the files they do not select appended in random order.

3.3 Metric

An ordering is read from the top, whole file by whole file, until the next file would exceed the line budget. Coverage is the fraction of the key read. We evaluate at budgets of 250 to 32,000 lines in doublings and report the AUC, the mean coverage over the eight budgets; ties are re-drawn three times and averaged. Python source runs to 8.50 tokens per line in aggregate over the 121,075 distinct file versions (c1100k_base), so a budget of 8,000 lines is roughly 68,000 tokens. Restricting the AUC to budgets up to 8,000 lines leaves the leading orderings unchanged.

AUC has no intuitive scale, so we anchor it. On the matched population a random ordering scores 0.249 and the oracle 0.774 on the co-edited key (0.252 and 0.779 on the symbol key), so a difference of 0.05 AUC is about a tenth of the distance from random to oracle; the recommended

fusion covers 45% of that distance on the co-edited key and 41% on the symbol key. We also report lines read before half the key is covered (Table 17 in the appendix).

The rule that reading stops at the first file that overflows the budget is part of the frozen metric. Because it favours short files, Section 4.3 also scores every ordering under a rule that skips such a file and continues, and under one that charges each file one unit whatever its length.

3.4 Statistics

Task AUCs are averaged within a pull request first. Paired comparisons use the Wilcoxon signed-rank test with the repository as the unit — pull-request scores averaged within each of the 112 repositories — because pull requests cluster in repositories that differ in size, layout and issue-writing culture. Effect sizes are Vargha–Delaney A_{12} ; confidence intervals bootstrap over repositories. Holm correction is applied within families fixed by the design (Table 20): each key, unit, population and leakage stratum is one family of 325 tests among twenty-six orderings. Ranks are among the twenty-two ranked orderings, the oracle excluded, so the best ordering is first. Differences quoted in the text are from the repository-level Wilcoxon analysis: means of per-repository differences, which can differ in the third decimal from the pull-request means in tables. The mixed-model estimates of the same comparisons are in Table 21 and can differ slightly from them.

The unit matters for borderline comparisons: of the 1,076 comparisons significant after Holm correction with the pull request as the unit, 102 are not with the repository as the unit and 2 move the other way; no estimate changes sign. Every comparison a claim rests on is also re-estimated as a linear mixed model of the per-pull-request difference with a random intercept per repository (Table 21); on 40 of those 41 comparisons it reaches the same verdict. The exception is the hop walk against path matching on co-edited files, where the mixed model finds path matching better by 0.044 ($p_{\text{Holm}} < 10^{-3}$) and the repository-level test does not; no claim rests on that comparison being null.

4 RESULTS

4.1 RQ1: the naive comparison, and why it does not survive

Table 3 is the comparison as it is usually run: each key over its full population, no control. Under the co-edited key the five best orderings are four fusions and one lexical method; the best purely structural ordering is ninth. Under the symbol key the four best orderings are three structural orderings and one fusion, and the best lexical ordering is fourteenth. The individual reversals are large and significant: path matching beats the length-aware walk by 0.059 AUC on co-edited files ($p_{\text{Holm}} < 10^{-4}$) and loses to it by 0.100 on symbol files ($p_{\text{Holm}} < 10^{-7}$). Read naively, the choice of key decides which signal wins.

The structural half is a population effect. The two columns are scored on different pull requests: 454 of the symbol column’s pull requests are single-file changes, which have no co-edited key and are exactly where a seed’s import neighbourhood is most of what there is to find. On the 702 pull requests carrying both keys (Table 7), nothing moves on the co-edited key, while on the symbol key a walk along import edges falls from first to seventh (0.491 to 0.417) and path matching rises from fourteenth to eighth. The same comparison that gave the walk a 0.101 lead on the full corpus ($p_{\text{Holm}} < 10^{-7}$) gives +0.008 and $p_{\text{Holm}} = 1$ on the matched population; for the hop walk against path matching it gives +0.019 and $p_{\text{Holm}} = 1$. The three top places under both keys are then held by the same three fusions.

The lexical half is largely the issue naming the answer. A key file’s full path appears verbatim in the issue for 12.6% of tasks and its bare stem for 48.5%. A stem written as code — `core.py`,

Table 3. The uncontrolled comparison: mean AUC by key, pull request as the unit, with bootstrap 95% confidence intervals over repositories. Neither control is applied; the text shows that both halves of the reversal dissolve under them.

co_edited (708 PRs)				symbol (1,156 PRs)			
#	Method	Fam.	AUC [CI]	#	Method	Fam.	AUC [CI]
–	oracle	ref	0.773 [.750,.796]	–	oracle	ref	0.793 [.772,.813]
1	rff_pprpl_issue_path	fus	0.486 [.455,.517]	1	ppr_out_pl	str	0.491 [.463,.517]
2	rff_hops_issue_path	fus	0.480 [.448,.510]	2	ppr_und_pl	str	0.489 [.460,.517]
3	rff_hops_path	fus	0.457 [.425,.489]	3	hops_lines	str	0.477 [.447,.505]
4	rff_ppr_issue_path	fus	0.445 [.414,.475]	4	rff_hops_path	fus	0.476 [.449,.502]
5	path_issue	lex	0.434 [.405,.465]	5	rff_pprpl_issue_path	fus	0.474 [.445,.501]
6	bm25_issue	lex	0.407 [.379,.435]	6	ppr_out	str	0.458 [.430,.487]
7	bm25_seed	lex	0.393 [.362,.425]	7	rff_hops_issue_path	fus	0.457 [.429,.483]
8	rff_ppr_issue	fus	0.393 [.363,.425]	8	ppr_und	str	0.446 [.417,.475]
9	hops_lines	str	0.389 [.354,.423]	9	bfs_und	str	0.420 [.391,.447]
10	ppr_und_pl	str	0.371 [.338,.403]	10	rff_ppr_issue_path	fus	0.420 [.392,.448]
11	path_seed	lex	0.371 [.338,.404]	11	pagerank	glo	0.410 [.381,.439]
12	bfs_und	str	0.362 [.330,.395]	12	indegree	glo	0.398 [.371,.425]
13	ppr_und	str	0.357 [.325,.391]	13	rff_ppr_issue	fus	0.390 [.361,.421]
14	bm25_issue_pl	lex	0.354 [.318,.389]	14	path_issue	lex	0.382 [.352,.412]
15	same_dir	ref	0.329 [.298,.360]	15	path_seed	lex	0.365 [.336,.392]
16	ppr_out_pl	str	0.327 [.296,.358]	16	bm25_issue_pl	lex	0.355 [.324,.386]
17	ppr_out	str	0.312 [.282,.344]	17	bm25_issue	lex	0.340 [.311,.368]
18	ppr_in_pl	str	0.310 [.279,.341]	18	bm25_seed	lex	0.336 [.305,.365]
19	ppr_in	str	0.308 [.278,.339]	19	same_dir	ref	0.320 [.290,.349]
20	pagerank	glo	0.271 [.240,.301]	20	ppr_in_pl	str	0.277 [.248,.303]
21	indegree	glo	0.271 [.240,.301]	21	ppr_in	str	0.269 [.242,.295]
22	random	ref	0.248 [.220,.277]	22	random	ref	0.248 [.220,.276]

simbad/core, a dotted module path, inside backticks or a code block — we call an *explicit* mention; one written only as an ordinary word, an *incidental* one. Explicit mentions cover 40.2% of tasks. Removing them, together with the population control (Table 8), takes path matching on co-edited files from fifth to twelfth and pure structure from ninth to fifth; the hop walk over path matching goes from -0.007 under the full-path control ($p_{\text{Holm}} = 1$) to $+0.042$ under the primary control ($p_{\text{Holm}} = 0.60$), on 531 pull requests. The ordering that looked like the best single signal on this key becomes indistinguishable from one that never reads the issue. The issue does not name the seed file more often than the key files, so the seed is not the explanation: its full path appears in 10.3% of tasks.

Which proxy to believe. Where the two keys still disagree, a key annotated by people decides. On ContextBench's tasks it sides with the co-edited key: the ranking under the human key agrees with the co-edited key's at Kendall $\tau = 0.97$ and with the symbol key's at 0.59, and the length-aware walks that are first and fourth under the symbol key are tenth and fourteenth under the human one (Table 13). So the answer to RQ1 is no: once the populations are matched and the naming is controlled, neither signal family dominates, and the key the field already uses is the better of the two proxies. The rest of this section asks what, then, the issue is worth, and which ordering is safe.

4.2 RQ2: what the issue is worth, and how much is the names

Stratifying by whether the issue names a key file compares two kinds of issue as well as two degrees of leakage: a report containing a stack trace is plausibly longer and more tractable than a

one-line feature request. We therefore measure the issue’s value twice, once across tasks and once within them.

Across tasks: an issue-free twin. Each fusion is paired with a twin in which every issue-derived input is replaced by its seed-derived counterpart: `rpf_pprpl_issue_path` with `rpf_pprpl_seedpath`, which fuses the same walk with the same two BM25 rankings queried by the seed’s own path and contents. The twin blends the same number of lists and never reads the issue, so the difference is what the issue is worth, net of “a fusion just gets more input”. On the matched population the issue is worth +0.069 AUC to the fusion on the co-edited key and +0.055 on the symbol key. On the tasks whose issue names a key file as code that becomes +0.156 and +0.121; on the rest, +0.016 and +0.012, and neither survives correction.

Within task: redaction. We then take the *same* issue and delete what points at the key files, leaving the prose, the traceback and the length. What counts as “pointing” is a choice, so we make it three times. The *names* arm deletes only explicit mentions of the key files’ stems; the *paths* arm also deletes every component of their paths; the *symbols* arm also deletes every function, class and method name they define, each only where written as code. The names arm is a lower bound on leakage; the wider arms over-remove, since a directory called `models` is also ordinary vocabulary, so the three bracket the effect. Of the 970 tasks whose issues name a key file, the 848 in the matched population — 378 pull requests in 105 repositories — are scored. Removing just the names costs path matching 0.110 AUC on co-edited files and the fusion 0.061; the wider arms cost the fusion 0.084 and 0.096 (Table 4). On the issues that name nothing the names arm changes nothing, as it must, while the wider arms cost 0.016 to 0.018 there, which is the part they over-remove.

The decomposition. Both experiments run on the same population with the same unit, so they add up. On issues that name a key file the issue is worth +0.156 to the fusion on co-edited files; on the rest, +0.016. Of that +0.140 gap the names arm attributes 0.061 to the names; the widest arm, net of what it also removes where nothing is named, attributes 0.078 (on the symbol key, 0.050 to 0.065 of a +0.109 gap). So between two fifths and a half of the difference between the two kinds of issue is the naming itself, and the rest is a difference between the issues, or the tasks, that no redaction we could define reaches: even path matching, which reads only file paths, scores 0.393 on named-file tasks after the widest redaction against 0.333 elsewhere.

A better reader. Everything above reads the issue with BM25, so “what the issue is worth” could describe the retriever rather than the issue. The dense retriever reads it better: alone it beats BM25 over contents by 0.059 and 0.054, and in place of BM25 in the fusion it adds 0.018 on both keys (Table 10). On issues that name nothing, the dense fusion gains 0.033 from the issue on co-edited files, twice what the BM25 fusion gains and, unlike it, significant; on the symbol key it gains 0.017, not significantly. So part of the small value of unnamed issues was BM25’s. But it is still a fifth of what the same fusion gains where the issue names a file, 0.178, and the names account for a similar share: of the 0.145 stratum gap, 40 to 47 per cent against 43 to 56 per cent for BM25. We did not test a reader that takes in the issue as a whole, such as an LLM reranker; the redaction protocol applies to one unchanged.

What this licenses. Not that issue text is useless: an ordering that exploits a real report naming a real file exploits something that exists. What it shows is that a benchmark assembled from issue reports measures two regimes at once, in proportions that vary by corpus, and that they can be separated for the cost of a few regular expressions and one extra run. Any evaluation on SWE-bench-derived data that reports a gain from issue-aware retrieval without the stratum and the redaction reports their sum.

Table 4. What the issue is worth to the BM25 fusion (rrf_pprpl_issue_path) and the dense fusion (rrf_pprpl_dense_path), and what each redaction arm costs them, on tasks whose issue names a key file as code and on the rest. Worth is the fusion minus its issue-free twin. Matched population, repository as the unit; * survives Holm correction.

	co_edited		symbol	
	BM25	dense	BM25	dense
issue worth, named	+0.156*	+0.178*	+0.121*	+0.135*
issue worth, not named	+0.016	+0.033*	+0.012	+0.017
names arm, named	+0.061*	+0.058*	+0.050*	+0.048*
paths arm, named	+0.084*	+0.082*	+0.074*	+0.068*
symbols arm, named	+0.096*	+0.086*	+0.082*	+0.074*
symbols arm, not named	+0.018*	+0.018*	+0.017*	+0.017*

Table 5. The recommended fusion under every check: its rank (oracle excluded) and its AUC shortfall behind the best ordering, per key. Rows marked † were tested against predictions committed before scoring; the others are exploratory.

Check	Ordering	co_edited (or human key)		symbol	
		rank	shortfall	rank	shortfall
Main sample, matched population	rrf_pprpl_issue_path	1	.000	1	.000
Main sample, full population	rrf_pprpl_issue_path	1	.000	5	.017
Held-out halves, 200 splits	rrf_pprpl_issue_path	best in 195/198	.002	best in 198/198	.000
Skip an overflowing file	rrf_pprpl_issue_path	2	.002	1	.000
Charge per file	rrf_pprpl_issue_path	6	.055	4	.045
† Seven unseen repositories	rrf_pprpl_issue_path	2	.030	2	.008
† Human key (ContextBench)	rrf_pprpl_issue_path	2	.009	–	–
No free seed (dense seed)	rrf_pprpl_dense_path	1	.000	1	.000

4.3 RQ3: a robust ordering, and how far it travels

If the answer key is not known in advance — for a tool shipped to users it is not — the quantity to optimise is the worst a method does across keys, measured as its shortfall behind the best ordering under either key. On the matched population the best on that criterion is the fusion, the best ordering under both keys, which beats the best pure lexical ordering by 0.055 and 0.067 and the best pure structural one by 0.097 and 0.039. It is never significantly worse than the best pure ordering in any leakage stratum on the matched population. On the full symbol-key population it is: once issues that name a key file are removed, the length-aware walks beat it by 0.049 to 0.062 ($p_{\text{Holm}} < 10^{-4}$), because that population keeps the single-file pull requests. On those, the five best orderings are all structural, and for a change that touches one file the walk alone is the better choice.

The fusion was found after seeing results, so we tested it. Table 5 collects every test; Table 6 gives the eight predictions committed before scoring.

Held-out repositories. Across 200 random 56/56 splits of the repositories, a mechanical criterion applied to one half selects the fusion in 198, and in 195 of those it is the best ordering on the co-edited key of the other half, 198 of 198 on the symbol key (Table 14). This shows the choice is stable across our repositories, not that it generalises.

Table 6. The predictions committed before the external repositories (E) and the ContextBench tasks (G) were scored, and what they showed. Mixed model with a random intercept per repository, Holm within each prediction.

	Prediction	Observed	Outcome
E1	fusion in the top three non-oracle orderings under both keys, worst shortfall ≤ 0.030	rank 2 and 2; shortfall 0.030	pass
E2	fusion never significantly worse than four pure orderings, either key	smallest difference +0.058 ($p_{\text{Holm}} = 0.511$)	pass
E3	outward walk beats inward on the symbol key	+0.169 ($p = 0.001$)	pass
E4	names redaction costs the fusion under both keys; nothing where nothing is named	+0.030 ($p_{\text{Holm}} = 0.112$) and +0.033 ($p_{\text{Holm}} = 0.008$); control ≤ 0.003	fail
G1	fusion in the top three non-oracle orderings on the human key, shortfall ≤ 0.030	rank 2; shortfall 0.009	pass
G2	fusion never significantly worse than four pure orderings	smallest difference +0.086 ($p_{\text{Holm}} < 0.001$)	pass
G3	outward walk beats inward	+0.083 ($p = 0.006$)	pass
G4	Kendall $\tau \geq 0.5$ between the human key's ranking and each proxy's	symbol 0.59, co-edited 0.97	pass

Seven unseen repositories. Seven repositories pass all our sampling rules and were left out only because their tasks come from SWE-Gym and SWE-bench Pro. We drew 140 pull requests from them by the same rules and committed four predictions before building anything. Three hold: the fusion is second under both keys, behind only another fusion built on the hop walk, with a worst shortfall of 0.030 — at the limit we set, not inside it — it is never significantly worse than a pure ordering, and walking inward is as costly as in the main sample. The fourth fails: deleting the names costs the fusion 0.030 on the co-edited key with $p_{\text{Holm}} = 0.11$, on only 33 matched pull requests in six repositories, so the causal effect of the names alone is not established there. The wider arms cost 0.086 and 0.090: in these larger repositories issues point at files through their directories more than their names.

A human-annotated key. On ContextBench's tasks all four predictions hold. The fusion is the second-best ordering on the human key, 0.009 behind a fusion of the same family, and beats each of four named pure orderings by 0.086 to 0.128 ($p_{\text{Holm}} \leq 0.008$). Its annotators worked from the fix, and 108 pull requests is a small set, but it is the only key in this paper that no part of our pipeline built.

Other cost rules. Under the frozen rule, preferring short files is close to a knapsack heuristic for the metric, so the length-aware walk could owe its place to the rule. Under a rule that skips a file that does not fit and keeps reading, the fusion is the best ordering on the symbol key and 0.002 behind the fusion built on the hop walk on the co-edited key; but the margin length-awareness buys shrinks by about sixty per cent (the two fusions differing only in it separate by +0.015 and +0.021 instead of +0.042 and +0.050). Under a rule that charges each file one unit, length-awareness is a cost, and the unscaled fusion `rfr_ppr_issue_path` is best under both keys. A fusion is the best ordering under every rule; whether its walk should favour short files depends on whether a context is charged by size, as a token window is, or by the number of files opened (Table 11).

Without a free seed. An agent is not handed the file to change. For each pull request we let the issue pick the seed — the top file of the dense retriever — walk from that guess and fuse, scoring

against every edited file (the true seeds included, so that finding the file is part of the task) and against the symbol key. The guess is the right file in only 34% of pull requests, but walking from it and fusing with the dense reader is the best ordering under both keys, at 0.516 and 0.475 (Table 12). It beats the dense retriever alone by 0.023 on the edited files ($p_{\text{Holm}} < 10^{-3}$) and 0.058 on the symbol key, and ties it where the issue names no edited file (-0.001). Seeding the walk with a BM25 guess instead, right in 41% of pull requests, does worse: the reader that picks the seed should be the one that reads the issue in the fusion.

Published systems' selectors. Scored as orderings on the matched population, the context selectors of Aider, RepoGraph and LocAgent rank between tenth and twentieth. The recommended fusion beats each by 0.115 to 0.176 on co-edited files and 0.084 to 0.155 on symbol files ($p_{\text{Holm}} < 10^{-4}$). Aider's repository map and RepoGraph's two-hop ego graph also fall below the plain length-aware walk; LocAgent's typed search ties it on co-edited files. These are the selectors alone, reimplemented from the papers without the language model each system wraps around them, so the comparison says what their graph component contributes as a reading order, not how well the systems perform.

The fusion's own parameter. RRF's k was fixed at 60 without testing; at $k = 10$ and $k = 200$ the fusion's AUC moves by at most 0.007 on the co-edited key and 0.010 on the symbol key.

4.4 RQ4: what the import graph contributes

The two-hop neighbourhood is half the repository. 86.7% of symbol-key files and 88.1% of co-edited files lie within two undirected hops of the seed (Table 15), which reads as strong evidence for graph retrieval. But the median two-hop ball holds 68 files and about 21,600 lines, more than five times a 4,000-line budget; measured against its own repository it is a median 58.5% of it (a mean of 56%), with a median precision of 3.4%, and it lifts recall over a random sample of the same size by a factor of only 1.57. It is not a filter: it says the files you need are somewhere in half the repository, which is where the *ordering* decides everything.

Direction: reversing a walk is costly, discarding it is free. Walking along import edges, towards what the seed depends on, places ppr_out_pl first on the symbol key at 0.491; walking against them collapses to 0.277, twentieth, a gap of 0.215 on the full population ($p_{\text{Holm}} < 10^{-14}$). The undirected walk loses almost nothing (0.489). On the co-edited key every directed walk ranks below same_dir, which consults no graph at all. The symbol key is built by resolving names along imports in one direction, so a walk in that direction measures something close to the key's construction.

How much is a repository prior. A query-independent PageRank, which never looks at the seed or the issue, reaches 0.410 on symbol files against 0.491 for the best personalised walk — 67% of the walk's margin over random — but only 0.271 on co-edited files, barely above random (0.248), where the walk beats it by 0.101. Knowing which file is changing matters a great deal for predicting what else was changed and much less for what the code calls.

The signals find different files. At $k = 20$, 24.4% of co-edited files are found only by the lexical family and 9.5% only by the structural family; on symbol files the asymmetry flips (20.0% against 16.5%, Table 16). This complementarity is why fusing them works; where the issue names a key file, much of the lexical side's share is that naming (Section 4.2).

What the human key says about the graph. On the gold files a fix reads but does not edit — the context a graph walk is supposed to be best at finding — walks and issue-only retrievers are indistinguishable on ContextBench's 85 pull requests that have any (differences of 0.006 to 0.012,

$p_{\text{Holm}} = 1$), and the fusion beats the issue-only orderings by 0.082 and 0.084 ($p_{\text{Holm}} \leq 0.008$). This analysis was added after the predictions were scored.

The budget is rarely the constraint. An oracle fits the complete key within 8,000 lines for 97.5% of pull requests, where the best ordering completes 58.5% on the co-edited key and 51.0% on the symbol key (Table 18). The oracle reads only key files, so the forty-point gap is an upper bound on what better ordering could add, not an estimate; what it establishes is that an ordering that fails to cover the key at this budget fails because of the order.

5 DISCUSSION

For benchmarks. The field's default key, the files the gold patch touched, is the better of our two proxies: a human-annotated key agrees with it, not with the key built from imports. The field's omission is elsewhere. Two tasks in five in this corpus have an issue that names a key file as code, and an issue-aware method's score mixes that regime with the other. The control costs a regular expression over text the benchmark already ships; the redaction costs one extra run. We think retrieval results on issue-derived benchmarks should report the naming rate, the stratified result and the redaction, as routinely as the corpus size.

For tools. Fuse a walk from the file being changed with issue retrieval, read the issue as well as you can — a dense retriever beats BM25 here — and make the walk favour short files only if the context is charged by size. For a change confined to one file, the walk alone is the better choice. Without a known file, let the same reader pick the starting point. The two-hop neighbourhood is not a useful filter on its own, and direction matters only in that walking against imports is costly.

What the baselines do not show. Our orderings are deliberately simple — BM25, PageRank, one embedding model — and the systems' selectors are reimplemented from their papers, not run end to end. A skeptic can read the fusion result as two weak complementary signals beating each alone. We agree that is what it is; its value is that it is robust, cheap and survives every test we could devise. The methodological contribution does not depend on it: the stratification and redaction protocol applies to any reader of the issue, including an LLM reranker, which we did not test.

Scope, and what remains. Every experiment in this paper runs on a CPU without paid model access, and within that budget we went as far as we could: two proxy keys, a human key, unseen repositories, a dense retriever, three systems' selectors, and a condition without the free seed. What it cannot show is whether better retrieval raises the rate at which an agent resolves the issue. That needs model calls at scale, and Appendix C specifies the experiment in advance so that it can be run as it stands. Its analysis would inherit the protocol this paper introduces — stratifying by naming, the issue-free twin, the redaction, the repository as the unit — which is the contribution we offer here.

Recommendations. (1) Report retrieval results per answer key, and prefer the patch key to an import-derived one. (2) Report and stratify by whether the query names the answer files as code, and run the redaction for whatever reads the issue. (3) Compare an issue-aware method against the same method with a seed-derived query. (4) Match populations before comparing across keys. (5) Cluster tests by repository. (6) State how reading is charged.

6 THREATS TO VALIDITY

Construct. Neither proxy key is ground truth about what a developer needed to read. The co-edited key conflates files changed with files read; the symbol key is circular with the structural orderings; the human key covers 108 pull requests and records what the fix needed as reconstructed

by annotators who saw it. Coverage assumes whole files are read; the alternative cost rules show which conclusions depend on that.

Internal. The graph builder misses dynamic imports and plugin registration. The symbol key inherits Jedi's resolution failures, which depend on the environment: rebuilding the keys of all 1,886 sampled pull requests on a second machine left every co-edited key identical but changed the symbol key of 41 pull requests, usually by one file. Re-scoring moves no AUC by more than 0.003 and changes no conclusion; in the held-out splits of Section 4.3 the fusion is then selected in 194 of 200 splits instead of 198, and confirmed on the co-edited key in 191 of those instead of 195. The paper reports the keys as first built. The leakage classifier is a regular expression and will mislabel some mentions, which is why we report three nested strata and three redaction arms. Leakage we did not think to redact — a paraphrase of a function's behaviour — would sit in the residual, so the leakage share is bounded below more firmly than above.

External. Python only, file granularity only, and pull requests from benchmarks that select for reproducible test-verified fixes, which may be better documented than average and so flatter issue-text methods. Very large repositories are under-represented; the pattern of Table 3 is stable across size quartiles (Table 19). The external set is seven repositories and one of its four predictions failed.

Conclusion. The fusion family was chosen after seeing results. The held-out splits, the external repositories and the human key test it, but none undoes that it was chosen by us, from five candidates, on these data.

7 CONCLUSION

What belongs in an agent's context window, given the file it is changing? Compared naively, import-graph proximity and issue-text matching trade places depending on the answer key. That reversal is the comparison, not the signals: one half is the different pull requests the two keys cover, the other the issues that name their files, and a human-annotated key sides with the key the field already uses. What survives is a protocol and a recommendation. Issue-aware retrieval is worth +0.156 AUC where the issue names a changed file against a non-significant +0.016 elsewhere, and about half of that difference is the naming itself; benchmarks should report the two regimes separately. And fusing a walk from the file with issue retrieval is robust — across keys, unseen repositories, a human key, cost rules and the loss of the free seed — except for single-file changes, where the walk alone is better.

DATA AVAILABILITY

The replication package contains the graphs, tasks, keys, issue bags, orderings and scores; the external and ContextBench trees with the predictions committed before they were scored; the scripts that compute the dense embeddings, which are regenerated rather than shipped; and every analysis script. A checker recomputes every number quoted in the text from the result files and fails if any sentence disagrees. It will be archived at a permanent identifier on acceptance.

USE OF AI TOOLS

Claude (Anthropic) was used to write the analysis and method code, to run the experiments and to draft this manuscript. All numbers are produced by scripts in the replication package and checked mechanically against the text; the author verified the design, the claims and the text. Bibliographic entries were checked against arXiv and publisher listings, and the evaluation section of every system in Table 1 was read directly.

REFERENCES

- [1] Aider. 2023. Building a Better Repository Map with Tree Sitter. <https://aider.chat/2023/10/22/repomap.html>. Blog post, 22 October 2023. Accessed 2026-09-23.
- [2] Reem Aleithan, Haoran Xue, Mohammad Mahdi Mohajer, Elijah Nnorom, Gias Uddin, and Song Wang. 2024. SWE-Bench+: Enhanced Coding Benchmark for LLMs. arXiv:2410.06992 [cs.SE]
- [3] Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. 2025. SWE-rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation of Software Engineering Agents. arXiv:2505.20411 [cs.SE]
- [4] Shawn A. Bohnert and Robert S. Arnold (Eds.). 1996. *Software Change Impact Analysis*. IEEE Computer Society Press, Los Alamitos, CA.
- [5] Dong Chen, Shaoxin Lin, Muhan Zeng, Daoguang Zan, Jian-Gang Wang, Anton Cheshkov, Jun Sun, Hao Yu, Guoliang Dong, Artem Aliev, Jie Wang, Xiao Cheng, Guangtai Liang, Yuchi Ma, Pan Bian, Tao Xie, and Qianxiang Wang. 2024. CodeR: Issue Resolving with Multi-Agent and Task Graphs. arXiv:2406.01304 [cs.SE]
- [6] Zhaoling Chen, Robert Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. 2025. LocAgent: Graph-Guided LLM Agents for Code Localization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Vienna, Austria, 8697–8727. <https://doi.org/10.18653/v1/2025.acl-long.426> Introduces the Loc-Bench benchmark. arXiv:2503.09089.
- [7] Bogdan Dit, Meghan Revelle, Malcom Gethers, and Denys Poshyvanyk. 2013. Feature Location in Source Code: A Taxonomy and Survey. *Journal of Software: Evolution and Process* 25, 1 (2013), 53–95. <https://doi.org/10.1002/smr.567>
- [8] Kang He and Kaushik Roy. 2026. SWE-Adept: An LLM-Based Agentic Framework for Deep Codebase Analysis and Structured Issue Resolution. arXiv:2603.01327 [cs.SE]
- [9] Zhonghao Jiang, Xiaoxue Ren, Meng Yan, Wei Jiang, Yong Li, and Zhongxin Liu. 2025. Issue Localization via LLM-Driven Iterative Code Graph Searching. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. Preprint titled “CoSIL: Software Issue Localization via LLM-Driven Code Repository Graph Searching”; arXiv:2503.22424.
- [10] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world GitHub Issues?. In *International Conference on Learning Representations (ICLR)*. arXiv:2310.06770. SWE-bench Lite is released with this benchmark.
- [11] Amy J. Ko, Brad A. Myers, Michael J. Coblenz, and Htet Htet Aung. 2006. An Exploratory Study of How Developers Seek, Relate, and Collect Relevant Information during Software Maintenance Tasks. *IEEE Transactions on Software Engineering* 32, 12 (Dec. 2006), 971–987. <https://doi.org/10.1109/TSE.2006.116>
- [12] Han Li, Letian Zhu, Bohan Zhang, Rili Feng, Jiaming Wang, Yue Pan, Earl T. Barr, Federica Sarro, Zhaoyang Chu, and He Ye. 2026. ContextBench: A Benchmark for Context Retrieval in Coding Agents. arXiv:2602.05892 [cs.SE]
- [13] Shanchao Liang, Spandan Garg, and Roshanak Zilouchian Moghaddam. 2025. The SWE-Bench Illusion: When State-of-the-Art LLMs Remember Instead of Reason. arXiv:2506.12286 [cs.SE] ICSE-SEIP 2026.
- [14] Wei Liu, Chao Peng, Pengfei Gao, Aofan Liu, Wei Zhang, Haiyan Zhao, and Zhi Jin. 2025. GraphLocator: Graph-guided Causal Reasoning for Issue Localization. arXiv:2512.22469 [cs.SE]
- [15] Yingwei Ma, Qingping Yang, Rongyu Cao, Binhua Li, Fei Huang, and Yongbin Li. 2025. Alibaba LingmaAgent: Improving Automated Issue Resolution via Comprehensive Repository Exploration. In *Companion Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion '25), Industry Track*. <https://doi.org/10.1145/3696630.3728549> arXiv:2406.01422 (v1 titled “How to Understand Whole Software Repository?”).
- [16] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. 2025. RepoGraph: Enhancing AI Software Engineering with Repository-Level Code Graph. In *International Conference on Learning Representations (ICLR)*. arXiv:2410.14684.
- [17] Bowen Qin and Yi Xie. 2026. Agent Retrieval Bench: Evaluating Repository Context Retrieval for Coding Agents. arXiv:2607.24882 [cs.SE]
- [18] Revanth Gangi Reddy, Tarun Suresh, JaeHyeok Doo, Ye Liu, Xuan Phi Nguyen, Yingbo Zhou, Semih Yavuz, Caiming Xiong, Heng Ji, and Shafiq Joty. 2025. SweRank: Software Issue Localization with Code Ranking. arXiv:2505.07849 [cs.SE]
- [19] Martin P. Robillard. 2008. Topology Analysis of Software Dependencies. *ACM Transactions on Software Engineering and Methodology* 17, 4 (Aug. 2008), 18:1–18:36. <https://doi.org/10.1145/13487689.13487691>
- [20] Lintang Sutawika, Aditya Bharat Soni, Bharath Sriraam R R, Apurva Gandhi, Taha Yassine, Sanidhya Vijayvargiya, Yuchen Li, Xuhui Zhou, Yilin Zhang, Leander Melroy Maben, and Graham Neubig. 2026. CodeScout: An Effective Recipe for Reinforcement Learning of Code Search Agents. arXiv:2603.17829 [cs.SE]

- [21] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying LLM-Based Software Engineering Agents. *Proceedings of the ACM on Software Engineering* 2, FSE (July 2025), FSE037. <https://doi.org/10.1145/3715754> Agentless. arXiv:2407.01489.
- [22] Chengxing Xie, Bowen Li, Chang Gao, He Du, Wai Lam, Difan Zou, and Kai Chen. 2025. SWE-Fixer: Training Open-Source LLMs for Effective and Efficient GitHub Issue Resolution. In *Findings of the Association for Computational Linguistics: ACL 2025*. 1123–1139. <https://doi.org/10.18653/v1/2025.findings-acl.62> arXiv:2501.05040.
- [23] Boyang Yang, Jiadong Ren, Shunfu Jin, Yang Liu, Feng Liu, Bach Le, and Haoye Tian. 2025. Enhancing Repository-Level Software Repair via Repository-Aware Knowledge Graphs. arXiv:2503.21710 [cs.SE] KGCompass. An earlier version was titled “KGCompass: Knowledge Graph Enhanced Repository-Level Software Repair”.
- [24] Annie T. T. Ying, Gail C. Murphy, Raymond Ng, and Mark C. Chu-Carroll. 2004. Predicting Source Code Changes by Mining Change History. *IEEE Transactions on Software Engineering* 30, 9 (Sept. 2004), 574–586. <https://doi.org/10.1109/TSE.2004.52>
- [25] Linghao Zhang, Shilin He, Chaoyun Zhang, Yu Kang, Bowen Li, Chengxing Xie, Junhao Wang, Maoquan Wang, Yufan Huang, Shengyu Fu, Elsie Nallipogu, Qingwei Lin, Yingnong Dang, Saravan Rajmohan, and Dongmei Zhang. 2025. SWE-bench Goes Live! arXiv:2505.23419 [cs.SE] NeurIPS 2025 Datasets and Benchmarks Track (per the authors' repository).
- [26] Shaoqiu Zhang, Yuhang Wang, Jialiang Liang, Yuling Shi, Wenhao Zeng, Maoquan Wang, Shilin He, Ningyuan Xu, Siyu Ye, Kai Cai, and Xiaodong Gu. 2026. SWE-Explore: Benchmarking How Coding Agents Explore Repositories. arXiv:2606.07297 [cs.SE]
- [27] Thomas Zimmermann, Peter Weißgerber, Stephan Diehl, and Andreas Zeller. 2004. Mining Version Histories to Guide Software Changes. In *Proceedings of the 26th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, Edinburgh, United Kingdom, 563–572.

A SUPPORTING TABLES

Table 7. The two keys re-scored on the 702 pull requests that carry both, top ten. Δ is the change in rank against Table 3.

Method	Fam.	AUC	#	Δ
oracle	ref	.774	0	–
rrf_pprpl_issue_path	fus	.487	1	–
rrf_hops_issue_path	fus	.480	2	–
rrf_hops_path	fus	.457	3	–
rrf_ppr_issue_path	fus	.445	4	–
path_issue	lex	.435	5	–
bm25_issue	lex	.409	6	–
rrf_ppr_issue	fus	.394	7	–1
bm25_seed	lex	.393	8	+1
hops_lines	str	.389	9	–
oracle	ref	.779	0	–
rrf_pprpl_issue_path	fus	.470	1	–4
rrf_hops_path	fus	.456	2	–2
rrf_hops_issue_path	fus	.454	3	–4
ppr_und_pl	str	.436	4	+2
hops_lines	str	.425	5	+2
rrf_ppr_issue_path	fus	.421	6	–4
ppr_out_pl	str	.417	7	+6
path_issue	lex	.404	8	–6
ppr_und	str	.395	9	+1

Table 8. Both controls at once: AUC and rank under three nested leakage strata, each on the pull requests within it that carry both keys.

Method	no full path		no explicit mention		no path or stem	
	co_edited	symbol	co_edited	symbol	co_edited	symbol
rrf_pprpl_issue_path	.468 (1)	.456 (1)	.434 (1)	.430 (3)	.417 (2)	.414 (4)
rrf_hops_path	.441 (3)	.445 (2)	.410 (3)	.421 (5)	.392 (3)	.404 (6)
ppr_out_pl	.325 (16)	.419 (6)	.335 (15)	.427 (4)	.331 (15)	.423 (3)
ppr_und_pl	.374 (10)	.440 (4)	.378 (9)	.445 (1)	.373 (9)	.441 (1)
hops_lines	.390 (8)	.427 (5)	.394 (5)	.435 (2)	.387 (5)	.429 (2)
path_issue	.408 (5)	.383 (10)	.363 (12)	.343 (18)	.339 (13)	.320 (19)
bm25_issue	.397 (6)	.357 (14)	.384 (6)	.346 (15)	.385 (8)	.346 (16)
pagerank	.274 (20)	.356 (15)	.281 (20)	.360 (13)	.285 (20)	.361 (12)
<i>pull requests</i>	655	655	531	531	487	487

Table 9. Redaction by method: AUC on the issue as written and the change under each arm, on the matched population within each stratum; * marks $p_{\text{Holm}} < 0.05$ within an arm.

Method	co_edited				symbol			
	AUC	names	paths	symbols	AUC	names	paths	symbols
<i>issue names a key file as code (378 pull requests in 105 repositories)</i>								
path_issue	.550	-0.110*	-0.148*	-0.157*	.498	-0.092*	-0.125*	-0.134*
bm25_issue	.452	-0.009*	-0.011*	-0.026*	.408	-0.006*	-0.008*	-0.021*
rrf_hops_path	.529	-0.072*	-0.097*	-0.105*	.515	-0.059*	-0.082*	-0.089*
rrf_pprpl_issue_path	.566	-0.061*	-0.084*	-0.096*	.537	-0.050*	-0.074*	-0.082*
<i>issue names no key file as code (531 pull requests in 109 repositories)</i>								
path_issue	.363	-0.000	-0.025*	-0.030*	.347	-0.002	-0.026*	-0.028*
bm25_issue	.376	-0.000	-0.003*	-0.006*	.341	-0.000	-0.002*	-0.004*
rrf_hops_path	.415	+0.001	-0.017*	-0.019*	.426	-0.001	-0.020*	-0.019*
rrf_pprpl_issue_path	.434	+0.002	-0.016*	-0.018*	.433	+0.000	-0.018*	-0.017*

Table 10. The dense retriever on the matched population: AUC and the rank each ordering would take if inserted alone into Table 3's ranking.

Ordering	co_edited	symbol
rrf_pprpl_issue_path_dense	.516 (1)	.480 (1)
rrf_pprpl_dense_path	.505 (1)	.488 (1)
rrf_pprpl_issue_path	.487 (1)	.470 (1)
rrf_pprpl_dense	.474 (3)	.476 (1)
dense_issue	.473 (3)	.422 (6)
path_issue	.435 (5)	.404 (8)
bm25_issue	.409 (6)	.367 (13)
dense_seed	.398 (7)	.364 (15)
bm25_seed	.393 (8)	.353 (16)

Table 11. AUC (and rank) of the leading orderings under three cost rules, matched population. *Stop*: the frozen rule. *Skip*: an overflowing file is passed over. *Files*: one unit per file.

Ordering	co_edited			symbol		
	stop	skip	files	stop	skip	files
rrf_pprpl_issue_path	.487 (1)	.519 (2)	.595 (6)	.470 (1)	.517 (1)	.571 (4)
rrf_ppr_issue_path	.445 (4)	.505 (3)	.649 (1)	.421 (6)	.496 (3)	.616 (1)
rrf_hops_issue_path	.480 (2)	.522 (1)	.631 (2)	.454 (3)	.509 (2)	.595 (2)
rrf_hops_path	.457 (3)	.479 (4)	.488 (9)	.456 (2)	.492 (4)	.483 (8)
hops_lines	.389 (9)	.399 (9)	.381 (14)	.425 (5)	.447 (8)	.410 (15)
ppr_und_pl	.372 (10)	.379 (13)	.268 (20)	.436 (4)	.448 (7)	.319 (19)
ppr_und	.358 (13)	.396 (10)	.497 (7)	.395 (9)	.456 (6)	.531 (7)
path_issue	.435 (5)	.456 (6)	.490 (8)	.404 (8)	.435 (10)	.452 (11)
bm25_issue	.409 (6)	.460 (5)	.615 (3)	.367 (13)	.418 (13)	.553 (5)

Table 12. Without a free seed: mean AUC per pull request. Suffix *_ds*: walked from the dense retriever's top file; otherwise from the BM25 guess. *Edited*: every edited file, true seeds included; each key split by whether the issue names an edited file as code.

Ordering	edited			symbol		
	all	named	not	all	named	not
rrf_pprpl_dense_path_ds	.516	.563	.469	.475	.503	.446
rrf_pprpl_issue_path_ds	.507	.553	.460	.461	.491	.431
rrf_pprpl_dense_path	.500	.553	.446	.458	.493	.423
rrf_pprpl_issue_path	.480	.533	.427	.440	.478	.403
dense_issue	.495	.521	.469	.417	.434	.400
issue-only fusion (picks the seed)	.471	.524	.417	.396	.436	.356
path_issue	.458	.531	.383	.397	.451	.342
bm25_issue	.417	.429	.405	.355	.369	.341
ppr_und_pl_ds	.409	.435	.383	.445	.456	.434
ppr_und_pl	.404	.448	.361	.429	.456	.402
random	.237	.239	.235	.240	.239	.240
oracle	.758	.763	.753	.760	.762	.758

Table 13. ContextBench tasks: AUC (and rank) under the human key, its read-only part (gold files the fix does not edit; exploratory), and the two proxy keys.

Ordering	gold	read-only	co_edited	symbol
rrf_hops_issue_path	.379 (1)	.347 (1)	.436 (1)	.343 (3)
rrf_pprpl_issue_path	.370 (2)	.341 (2)	.423 (2)	.349 (2)
rrf_hops_path	.345 (4)	.317 (4)	.387 (3)	.338 (5)
bm25_issue	.283 (6)	.259 (9)	.333 (5)	.217 (16)
path_issue	.280 (8)	.248 (12)	.322 (6)	.228 (14)
hops_lines	.275 (9)	.274 (6)	.281 (9)	.309 (6)
ppr_und_pl	.261 (10)	.267 (7)	.261 (10)	.352 (1)
ppr_out_pl	.230 (14)	.249 (11)	.202 (15)	.339 (4)
pagerank	.180 (17)	.192 (17)	.159 (19)	.261 (11)
random	.103 (22)	.104 (22)	.104 (22)	.095 (22)
oracle	.694	.699	.739	.731

Table 14. 200 random 56/56 splits of the repositories: how often the criterion selected each fusion, how often it was then best on the held-out half, and its mean shortfall there.

Method	selected	co_edited		symbol	
		best	gap	best	gap
rrf_pprpl_issue_path	198/200	195/198	.002	198/198	.000
rrf_hops_issue_path	2/200	0/2	.016	0/2	.024

Table 15. Where the key files sit relative to the seed, and the size of the two-hop ball (union of the two keys, 2,412 tasks).

Key files at hop distance	1	2	3+	unreachable
co_edited	39.0%	49.1%	9.1%	2.8%
symbol	45.3%	41.4%	9.1%	4.2%

The two-hop ball: 68 files, 21,646 lines, 56% of the repository
 recall 87.5%, precision 3.4%, recall lift over a same-sized random sample 1.57×
The union key: 3 files, 1,515 lines

Table 16. How each key file is reached within the top k of the structural family (ppr_und, hops_lines) and the lexical family (bm25_issue, path_issue).

Source	k	both	structure only	words only	neither
co_edited	10	32.2%	11.4%	30.1%	26.4%
co_edited	20	50.9%	9.5%	24.4%	15.3%
co_edited	50	76.5%	6.1%	13.3%	4.1%
symbol	10	28.4%	23.4%	20.5%	27.7%
symbol	20	47.2%	20.0%	16.5%	16.2%
symbol	50	72.9%	11.5%	9.8%	5.9%

Table 17. Median lines read before half, then all, of the key is covered.

Method	to half the key		to all of it	
	co_edited	symbol	co_edited	symbol
oracle	524	440	1,150	1,349
rrf_pprpl_issue_path	3,342	3,957	7,213	9,563
rrf_hops_path	3,962	3,715	8,073	9,636
ppr_out_pl	8,655	3,924	15,396	12,386
ppr_und_pl	6,155	3,483	11,644	11,303
hops_lines	5,620	4,080	11,454	11,582
path_issue	4,838	6,638	11,072	14,504
bm25_issue	5,012	7,451	10,926	16,699
random	14,557	14,713	21,026	22,643

Table 18. Share of pull requests taken to complete coverage within 8,000 lines.

Ordering	co_edited (%)	symbol (%)
oracle	97.5	95.9
rrf_hops_issue_path	58.5	49.3
rrf_pprpl_issue_path	56.5	49.1
rrf_hops_path	54.5	50.3
rrf_ppr_issue_path	53.4	45.0
bm25_issue	51.8	36.9
rrf_ppr_issue	50.6	44.6
bm25_seed	48.8	36.6
path_issue	48.4	36.9
hops_lines	48.1	51.0
ppr_und_pl	45.1	50.0
ppr_und	45.0	50.1
bm25_issue_pl	39.8	33.9
path_seed	39.6	36.3
bfs_und	36.6	40.1
ppr_out_pl	35.4	50.1
ppr_out	34.5	49.7
ppr_in	34.0	22.8
ppr_in_pl	33.9	22.3
pagerank	31.9	43.4
indegree	28.2	38.9
same_dir	27.8	24.2
random	15.3	14.3

Table 19. Mean AUC (rank) by quartile of repository size in graph nodes: $Q1 \leq 73$, $Q2 \leq 131$, $Q3 \leq 246$, $Q4$ up to 2,487. Full corpus, no control.

Method	co_edited				symbol			
	Q1	Q2	Q3	Q4	Q1	Q2	Q3	Q4
rrf_pprpl_issue_path	.574 (1)	.487 (1)	.418 (1)	.369 (1)	.568 (2)	.476 (1)	.403 (4)	.375 (2)
rrf_hops_path	.552 (3)	.465 (3)	.397 (3)	.330 (4)	.562 (5)	.473 (2)	.409 (3)	.362 (4)
ppr_out_pl	.473 (15)	.359 (16)	.268 (15)	.168 (16)	.562 (4)	.450 (6)	.414 (2)	.372 (3)
ppr_und_pl	.497 (12)	.399 (8)	.316 (10)	.241 (11)	.568 (3)	.467 (3)	.423 (1)	.387 (1)
hops_lines	.517 (6)	.412 (6)	.329 (8)	.244 (10)	.569 (1)	.454 (5)	.402 (5)	.343 (7)
path_issue	.537 (5)	.437 (4)	.372 (5)	.302 (5)	.515 (13)	.416 (9)	.324 (12)	.257 (14)
bm25_issue	.516 (7)	.402 (7)	.334 (6)	.287 (6)	.476 (17)	.367 (17)	.273 (17)	.233 (16)
pagerank	.442 (20)	.299 (21)	.199 (20)	.113 (21)	.521 (10)	.370 (16)	.333 (11)	.286 (12)

B CORRECTION FAMILIES AND MIXED-MODEL ESTIMATES

Table 20. Every Holm family. Paired families hold all 325 tests among twenty-six orderings (the twenty tested contenders, two issue-free twins and four names-redacted variants).

Tests	Population	Stratum	Keys	Tests per family
paired tests, pull request as unit	full	all	3	325
paired tests, pull request as unit	matched	all	3	325
paired tests, repository as unit	full	all	3	325
paired tests, repository as unit	full	explicit	2	325
paired tests, repository as unit	full	no explicit	2	325
paired tests, repository as unit	full	no full path	2	325
paired tests, repository as unit	full	no stem	2	325
paired tests, repository as unit	matched	all	3	325
paired tests, repository as unit	matched	explicit	2	325
paired tests, repository as unit	matched	no explicit	2	325
paired tests, repository as unit	matched	no full path	2	325
paired tests, repository as unit	matched	no stem	2	325
redaction, names arm	matched	by group	1	24
redaction, paths arm	matched	by group	1	24
redaction, symbols arm	matched	by group	1	24
mixed model, headline comparisons	both	several	1	41

Table 21. The comparisons claims rest on, re-estimated as a linear mixed model of the per-pull-request difference $a - b$ with a random intercept per repository. Holm-corrected as one family.

a	b	Key	Stratum	Pop.	PRs	Estimate [95% CI]	P_{Holm}
rrf_pprpl_issue_path	path_issue	co_edited	all	full	708	+0.053 [0.040,0.067]	$< 10^{-12}$
rrf_pprpl_issue_path	path_issue	co_edited	all	matched	702	+0.054 [0.040,0.067]	$< 10^{-12}$
rrf_pprpl_issue_path	path_issue	symbol	all	full	1156	+0.089 [0.075,0.104]	$< 10^{-30}$
rrf_pprpl_issue_path	path_issue	symbol	all	matched	702	+0.067 [0.054,0.079]	$< 10^{-23}$
ppr_out_pl	path_issue	co_edited	all	full	708	-0.105 [-0.125,-0.085]	$< 10^{-23}$
ppr_out_pl	path_issue	co_edited	all	matched	702	-0.106 [-0.126,-0.087]	$< 10^{-23}$
ppr_out_pl	path_issue	symbol	all	full	1156	+0.105 [0.082,0.127]	$< 10^{-17}$
ppr_out_pl	path_issue	symbol	all	matched	702	+0.012 [-0.012,0.037]	1
ppr_und_pl	path_issue	co_edited	all	full	708	-0.062 [-0.082,-0.042]	$< 10^{-7}$
ppr_und_pl	path_issue	co_edited	all	matched	702	-0.062 [-0.082,-0.043]	$< 10^{-7}$
ppr_und_pl	path_issue	symbol	all	full	1156	+0.103 [0.078,0.128]	$< 10^{-13}$
ppr_und_pl	path_issue	symbol	all	matched	702	+0.032 [0.007,0.056]	0.143
hops_lines	path_issue	co_edited	all	full	708	-0.044 [-0.064,-0.023]	$< 10^{-3}$
hops_lines	path_issue	co_edited	all	matched	702	-0.044 [-0.065,-0.024]	$< 10^{-3}$
hops_lines	path_issue	symbol	all	full	1156	+0.091 [0.067,0.115]	$< 10^{-11}$
hops_lines	path_issue	symbol	all	matched	702	+0.022 [-0.000,0.044]	0.610
rrf_pprpl_issue_path	ppr_out_pl	symbol	all	full	1156	-0.014 [-0.036,0.007]	1
rrf_pprpl_issue_path	ppr_out_pl	symbol	all	matched	702	+0.054 [0.032,0.076]	$< 10^{-4}$
rrf_pprpl_issue_path	ppr_out_pl	symbol	no explicit	full	871	-0.061 [-0.083,-0.040]	$< 10^{-6}$
rrf_pprpl_issue_path	ppr_out_pl	symbol	no explicit	matched	531	+0.000 [-0.024,0.024]	1
rrf_pprpl_issue_path	ppr_und_pl	symbol	all	full	1156	-0.013 [-0.036,0.009]	1
rrf_pprpl_issue_path	ppr_und_pl	symbol	all	matched	702	+0.035 [0.015,0.055]	0.008
rrf_pprpl_issue_path	ppr_und_pl	symbol	no explicit	full	871	-0.057 [-0.079,-0.035]	$< 10^{-5}$
rrf_pprpl_issue_path	ppr_und_pl	symbol	no explicit	matched	531	-0.016 [-0.037,0.005]	1
rrf_pprpl_issue_path	hops_lines	symbol	all	full	1156	-0.002 [-0.019,0.014]	1
rrf_pprpl_issue_path	hops_lines	symbol	all	matched	702	+0.045 [0.027,0.062]	$< 10^{-4}$
rrf_pprpl_issue_path	hops_lines	symbol	no explicit	full	871	-0.047 [-0.063,-0.032]	$< 10^{-7}$
rrf_pprpl_issue_path	hops_lines	symbol	no explicit	matched	531	-0.007 [-0.026,0.013]	1
hops_lines	path_issue	co_edited	no full path	matched	655	-0.016 [-0.037,0.004]	1
hops_lines	path_issue	co_edited	no explicit	matched	531	+0.034 [0.009,0.059]	0.103
hops_lines	path_issue	co_edited	no stem	matched	487	+0.050 [0.024,0.077]	0.004
ppr_out_pl	ppr_in_pl	symbol	all	full	1156	+0.215 [0.192,0.238]	$< 10^{-71}$
ppr_out_pl	ppr_in_pl	symbol	all	matched	702	+0.123 [0.104,0.143]	$< 10^{-33}$
rrf_pprpl_issue_path	rrf_pprpl_issue_path_rd	co_edited	explicit	matched	378	+0.063 [0.050,0.075]	$< 10^{-20}$
rrf_pprpl_issue_path	rrf_pprpl_issue_path_rd	co_edited	no explicit	matched	531	-0.002 [-0.005,0.001]	1
rrf_pprpl_issue_path	rrf_pprpl_issue_path_rd	symbol	explicit	matched	378	+0.049 [0.040,0.058]	$< 10^{-23}$
rrf_pprpl_issue_path	rrf_pprpl_issue_path_rd	symbol	no explicit	matched	531	-0.000 [-0.003,0.003]	1
rrf_pprpl_issue_path	rrf_pprpl_seedpath	co_edited	explicit	matched	378	+0.153 [0.130,0.176]	$< 10^{-36}$
rrf_pprpl_issue_path	rrf_pprpl_seedpath	co_edited	no explicit	matched	531	+0.020 [0.005,0.035]	0.133
rrf_pprpl_issue_path	rrf_pprpl_seedpath	symbol	explicit	matched	378	+0.116 [0.095,0.137]	$< 10^{-25}$
rrf_pprpl_issue_path	rrf_pprpl_seedpath	symbol	no explicit	matched	531	+0.012 [-0.003,0.027]	1

C A PROTOCOL FOR THE DOWNSTREAM TEST

Whether a context assembled by one of these orderings lets a model fix the issue requires running models, which we have not done; we specify the experiment here so that it can be run or contradicted. *Corpus*: the 1,162 scored pull requests, each with a benchmark test specification. *Conditions*, each truncated to the same token budget of 16,000 or 64,000 tokens: (i) the seed alone; (ii) seed plus ppr_und_pl; (iii) seed plus path_issue; (iv) seed plus the recommended fusion; (v) seed plus rrf_hops_path; (vi) seed plus the issue-free twin of (iv); (vii) the repository in a fixed canonical order; (viii) seed plus the key files, an upper reference. *Models*: two of clearly different capability, one API call per task and condition, no tools. *Outcome*: a task is resolved if the patch applies and the specified tests pass; conditions are compared with a mixed-effects logistic regression with a

random intercept per repository, McNemar tests alongside, Holm across condition pairs. *Pilot*: 150 tasks on the weaker model; if (viii) resolves under 10% or over 90%, the budget is adjusted and the adjustment reported. If (ii)–(vi) do not differ from (vii), retrieval quality does not transfer to repair at these budgets; if (iv) does not beat (vi), the issue does not earn its place downstream either.